Data Management for Machine Learning
(Merged - AIMLCZG529/DSECLZG529)
S2-25
Assignment I

Submission Date: 22.07.2026
Weightage: 20 Marks (EC1)

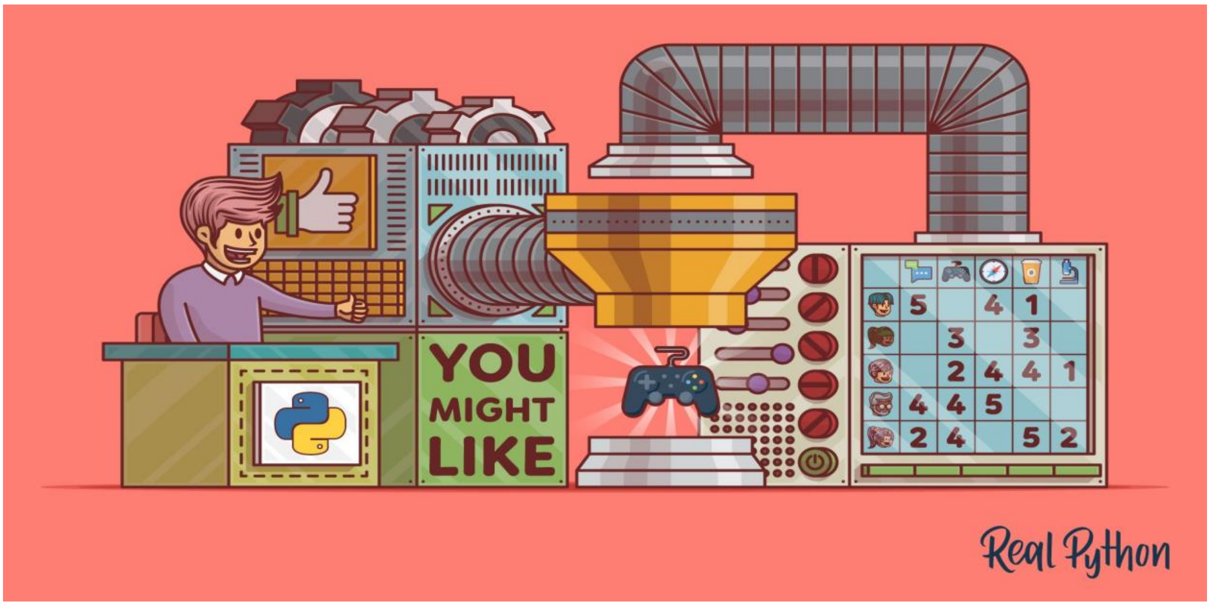
Title: *End-to-End Data Management Pipeline for a Recommendation System*

Objective

Design and implement a complete data management pipeline for a recommendation system. This pipeline should support data ingestion, validation, preparation, transformation, feature management, model training, and orchestration — aligned with the practices of modern data stacks and ML workflows.

Business Context

Online recommendation systems are fundamental to modern digital platforms such as e-commerce sites, OTT services, and music streaming applications. They personalize user experiences, improve engagement, and drive sales conversions.



Your client — **RecoMart**, an e-commerce startup — wants to build a **data-driven recommendation engine** to enhance customer engagement and cross-selling opportunities. The platform collects user behavior data from multiple sources:

- Web and mobile clickstream logs
- Transactional purchase history
- Product metadata from catalogs
- External APIs (e.g., sentiment or popularity scores)

RecoMart expects a scalable and maintainable **data management pipeline** to continuously process incoming data, curate features, and train/update models for generating personalized recommendations.

Scenario

As a **Data Engineer** in RecoMart's Data Platform Team, you are tasked to design and implement an **automated, modular data pipeline** supporting both batch and near-real-time ingestion. The goal is to ensure that the recommendation model always learns from fresh, validated, and well-structured data.

Tasks

1. Problem Formulation

- Define the **business problem** clearly (e.g., product recommendation to improve conversion rate).
- Identify the **key data sources** and their attributes (e.g., users, items, ratings, transactions).
- Define the **expected outputs** from the pipeline:
  - Clean datasets for Exploratory Data Analysis (EDA)
  - Engineered features for collaborative/content-based models
  - Deployable recommendation model and inference interface
- Specify **evaluation metrics** (e.g., Precision@K, Recall@K, NDCG).

Deliverables:

- A short report (PDF) outlining problem definition, objectives, data sources, and expected pipeline outcomes.

---

2. Data Collection and Ingestion

- Ingest at least **two types of data** (e.g., user interactions from CSV files and product data from REST APIs).
- Design ingestion scripts ensuring:
  - Automated and periodic data fetching
  - Error handling and retry mechanisms
  - Logging for monitoring and audit trails

Deliverables:

- Python scripts for ingestion
- Logs showing ingestion success/failure
- Folder/bucket structure for raw data

---

3. Raw Data Storage

- Store ingested data in a **local data lake or cloud storage** (e.g., AWS S3 or local filesystem).
- Use a structured folder layout partitioned by **source**, **type**, and **timestamp**.

Deliverables:

- Storage structure documentation
- Upload scripts or configuration files

---

4. Data Profiling and Validation

- Apply validation checks to ensure **data quality and completeness**:
  - Missing values, duplicate entries, schema mismatch
  - Range and format checks (e.g., rating scale 1–5)
- Generate a **data quality report** summarizing key metrics and issues.

Deliverables:

- Python code for automated validation (using `pandas`, `great_expectations`, or `pydeequ`)
- Data Quality Report (PDF)

---

5. Data Preparation

- Perform data cleaning and preprocessing:
  - Handle missing user-item interactions
  - Encode categorical attributes (e.g., product categories, user demographics)
  - Normalize numerical variables (e.g., prices, timestamps)
- Conduct exploratory analysis showing interaction distributions, item popularity, and sparsity patterns.

Deliverables:

- Jupyter notebook or script demonstrating cleaning and EDA
- Summary plots (histograms, heatmaps)
- Prepared dataset ready for transformation

---

6. Feature Engineering and Transformation

- Create features suitable for recommendation algorithms, such as:
  - User activity frequency
  - Average rating per user/item
  - Co-occurrence or similarity-based features
- Store transformed data in a structured database or warehouse.

Deliverables:

- SQL schema
- Transformation scripts
- Summary of feature logic

---

7. Feature Store

- Implement a simple **feature store** (using Feast or a custom metadata registry).
- Document feature names, sources, and transformations.

- Enable versioned retrieval for both training and inference.

Deliverables:

- Feature store configuration/code
- Feature metadata documentation
- Sample feature retrieval demonstration

---

8. Data Versioning and Lineage

- Use tools like **DVC** or **Git LFS** to version raw and transformed data.
- Track metadata such as data source, date of ingestion, and applied transformations.

Deliverables:

- Repository structure showing dataset versions
- Documentation of versioning workflow

---

9. Model Training and Evaluation

- Train at least one recommendation model:
  - Collaborative filtering (Matrix Factorization / SVD)
  - Content-based filtering
- Evaluate model using metrics.
- Store model metadata using **MLflow** or an equivalent tracking tool.

Deliverables:

- Training and evaluation script
- Model performance report
- Tracked model metadata (run IDs, parameters, metrics)

---

10. Pipeline Orchestration

- Automate the end-to-end data pipeline using **Airflow**, **Prefect**, or **Dagster**.
- Define DAGs for data ingestion → validation → preparation → transformation → feature store → model training.
- Include logging and monitoring for failures.

Deliverables:

- Orchestration DAG/code
- Screenshots or logs from the orchestration tool showing successful execution

---

Additional Instructions

- Ensure modular code structure and clear documentation.
- Maintain logs, error handling, and reproducibility practices.
- Provide a **short demo video (5–10 mins)** explaining the pipeline flow.

----

12. Submission Requirements

Students Group must submit the following deliverables:

Submission Format

- Submit a **single PDF report** containing the complete project documentation.

PDF Report Should Include

- **Project Title**
- **Team Member Details**
- **Problem Statement**
- **Objectives**
- **Methodology / Pipeline**
- **Implementation Details**
- **Results and Output Screenshots**
- **Conclusion and Future Scope**
- **Google Drive Link for the following**
- **Google Drive Link** to the **Video Walkthrough demonstrating** **(5–10 mins)** the complete end-to-end execution of the project
- **Google Drive Link** to the .zip file containing all project deliverables, including:
    - Source Code
    - Dataset(s) (if applicable)
    - Trained Model(s) (if applicable)
    - Required Documentation
    - Any additional supporting files

  (Google Drive Link should be Accessible to Any BITS ID)

Evaluation Rubric (20 Marks)

Component	Description	Weight (20 Marks)
Problem Formulation	Clear definition of objectives, data sources, and outputs	2
Data Pipeline Implementation	Ingestion, storage, validation, and transformation logic	5
Feature Store & Versioning	Effective management of features and data lineage	4
Model Training & Evaluation	Performance metrics and experiment tracking	4
Documentation & Demo	Clarity, completeness, and demonstration quality	5
